# SFWRENG-3BB4 Assignment 2

Matthew Farah, ⟨farahm11@mcmaster.ca⟩, 400468251

November 3, 2025

## Contents

## Question 1

Consider a group of $k$ philosophers. Each philosopher either thinks or eats cookies (one serving at a time) or drinks cola (one bottle at a time). The cookie dispenser has a capacity of $M$ servings, and the cola dispenser has a capacity of $N$ bottles. When a dispenser is empty a philosopher must wait until a servant refills it. If any dispenser is empty, the server refills it with either $M$ servings of cookies or $N$ bottles of cola, depending on which dispenser was empty. Refilling must be done as soon as possible, so they have priority over dispensing cookies and cola. There is no partial refilling of the dispensers. Assume that initially, both dispensers are full.

**1.a)** Model the behaviour of the system as an FSP. Write a safety property that, when composed with your system, will check if no cookies or cola are dispensed when their respective dispensers are empty. Compose this property with your system and verify that it holds.

**Solution:**

$$\texttt{PHILOSOPHER} = ($$
$$\quad \texttt{think} \rightarrow \texttt{PHILOSOPHER} \mid \texttt{get\_cookie} \rightarrow \texttt{eat\_cookie} \rightarrow \texttt{PHILOSOPHER}$$
$$\quad \mid \texttt{get\_cola} \rightarrow \texttt{drink\_cola} \rightarrow \texttt{PHILOSOPHER}$$
$$).$$
$$\texttt{const K} = 3.$$
$$\texttt{range Phil} = 1..K.$$
$$||\texttt{PHILOSOPHERS} = (\texttt{forall}[i : \texttt{Phil}]\ \texttt{phil}[i] : \texttt{PHILOSOPHER}).$$
$$\texttt{SERVANT} = (\texttt{fill\_cookies} \rightarrow \texttt{SERVANT} \mid \texttt{fill\_cola} \rightarrow \texttt{SERVANT}).$$
$$\texttt{const M} = 3.$$
$$\texttt{range Cookies} = 0..M.$$
$$\texttt{COOKIES} = \texttt{COOKIES}[0],$$
$$\texttt{COOKIES}[s : \texttt{Cookies}] = (\ \texttt{when}\ (s > 0)\ \texttt{get\_cookie} \rightarrow \texttt{COOKIES}[s-1] \mid \texttt{fill\_cookies} \rightarrow \texttt{COOKIES}[M]\ ).$$
$$\texttt{const N} = 2.$$
$$\texttt{range Cola} = 0..N.$$
$$\texttt{COLA} = \texttt{COLA}[0],$$
$$\texttt{COLA}[s : \texttt{Cola}] = (\ \texttt{when}\ (s > 0)\ \texttt{get\_cola} \rightarrow \texttt{COLA}[s-1] \mid \texttt{fill\_cola} \rightarrow \texttt{COLA}[N]\ ).$$
$$\texttt{property SAFETY\_COLA} = \texttt{SAFETY\_COLA}[N],$$
$$\texttt{SAFETY\_COLA}[s : \texttt{Cola}] = (\ \texttt{when}\ (s == 0)\ \texttt{fill\_cola} \rightarrow \texttt{SAFETY\_COLA}[N]\ ).$$
$$\texttt{property SAFETY\_COOKIES} = \texttt{SAFETY\_COOKIES}[M],$$
$$\texttt{SAFETY\_COOKIES}[s : \texttt{Cookies}] = (\ \texttt{when}\ (s == 0)\ \texttt{fill\_cookies} \rightarrow \texttt{SAFETY\_COOKIES}[M]\ ).$$
$$||\texttt{SYSTEM} = (\ \texttt{PHILOSOPHERS} \mid\mid \texttt{COOKIES} \mid\mid \texttt{COLA} \mid\mid \texttt{SERVANT} \mid\mid \texttt{SAFETY\_COLA} \mid\mid \texttt{SAFETY\_COOKIES}\ ).$$

**1.b)** Model the behaviour of the system as an elementary Petri net.

**Solution:**

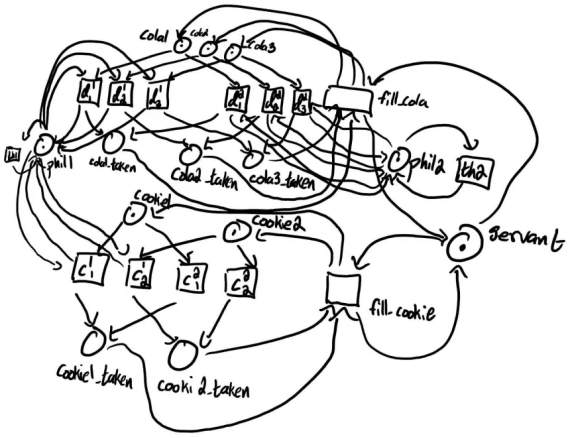


Figure 1: Elementary Petri net representation of the system.

**1.c)** Model the behaviour of the system as a place/transition Petri net.

**Solution:**

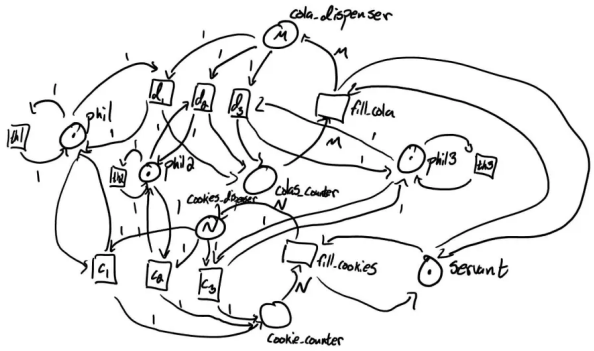


Figure 2: Place/transition Petri net representation of the system.

**1.d)** Model the behaviour of the system as a coloured Petri net.

**Solution:**

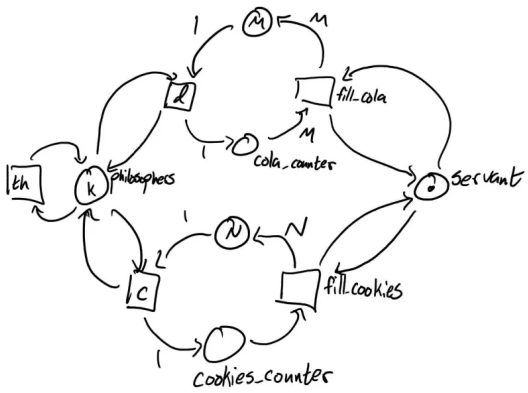


Figure 3: Coloured Petri net representation of the system.

**1.e)** Discuss the differences between the FSP and the various Petri net solutions.

**Solution:**

The primary difference between the FSP and Petri net solutions, besides the obvious fact that the Petri nets provide a graphical, as opposed to an algebraic, description of the problem, is that FSPs are much more difficult to perform rigorous error/proof checking on in comparison to Petri nets.

**1.f)** Implement the system in a Java program.

**Solution:**

Listing 1: Concurrent system for philosophers, cookie and cola dispensers

```
class Dispenser {
    private final String name;
    private final int capacity;
    private int amount;

    public Dispenser(String name, int capacity) {
        this.name = name;
        this.capacity = capacity;
        this.amount = capacity;
    }

    public synchronized void getItem(String philosopherName) throws
        InterruptedException {
        while (amount == 0) {
            wait();
        }
        amount--;
        notifyAll();
    }
```

```
    public synchronized void refill() {
        amount = capacity;
        notifyAll();
    }

    public synchronized boolean isEmpty() {
        return amount == 0;
    }
}

class Servant extends Thread {
    private final Dispenser cookies;
    private final Dispenser cola;

    public Servant(Dispenser cookies, Dispenser cola) {
        this.cookies = cookies;
        this.cola = cola;
    }

    @Override
    public void run() {
        while (true) {
            try {
                synchronized (this) {
                    if (cookies.isEmpty()) {
                        cookies.refill();
                    }
                    if (cola.isEmpty()) {
                        cola.refill();
                    }
                }
                Thread.sleep(100);
            } catch (InterruptedException e) {
                return;
            }
        }
    }
}

class Philosopher extends Thread {
    private final String name;
    private final Dispenser cookies;
    private final Dispenser cola;

    public Philosopher(String name, Dispenser cookies, Dispenser cola) {
        this.name = name;
        this.cookies = cookies;
        this.cola = cola;
    }

    @Override
    public void run() {
        try {
            while (true) {
                think();
                eatCookies();
                drinkCola();
            }
        } catch (InterruptedException e) {
```

```
        }
    }

    private void think() throws InterruptedException {
        Thread.sleep((int) (Math.random() * 1000));
    }

    private void eatCookies() throws InterruptedException {
        cookies.getItem(name);
        Thread.sleep((int) (Math.random() * 500));
    }

    private void drinkCola() throws InterruptedException {
        cola.getItem(name);
        Thread.sleep((int) (Math.random() * 500));
    }
}

public class PhilosopherSystem {
    public static void main(String[] args) {
        final int K = 5; // number of philosophers
        final int M = 3; // cookie capacity
        final int N = 2; // cola capacity

        Dispenser cookies = new Dispenser("cookies", M);
        Dispenser cola = new Dispenser("cola", N);

        Servant servant = new Servant(cookies, cola);
        servant.start();

        for (int i = 0; i < K; i++) {
            new Philosopher("Philosopher␣" + (i + 1), cookies, cola).start();
        }
    }
}
```

## Question 2

A museum allows visitors to enter through the east entrance and leave through its west exit. Arrivals and departures are signalled to the museum by turnstiles at the museum's entrance and exit. On opening, the museum director signals to the controller that the museum is open and then the controller permits both arrivals and departures. On closing, the director signals to the controller that the museum is closed, at which point only departures are permitted by the controller.

As an FSP, the museum system consists of the processes `EAST`, `WEST`, `CONTROL`, and `Director`.

**2.a)** Draw the structure diagram for the museum and provide an FSP description for each of the processes and their overall composition.

**Solution:**

**(II)** Draw the structure diagram.

The structure diagram for the museum can be given by:

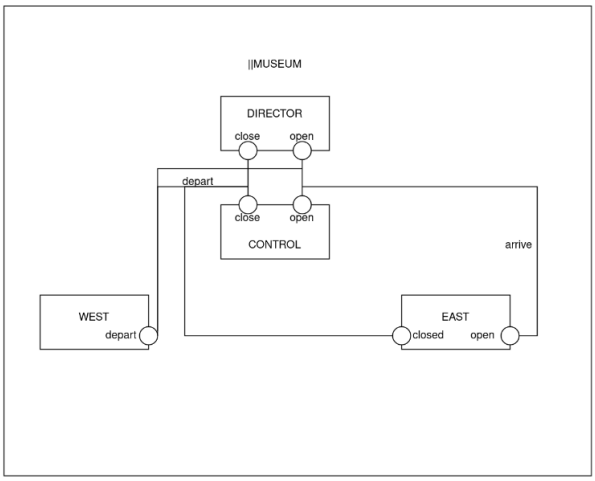


Figure 4: Structure diagram for the `MUSEUM` system.

**(III)** Provide an FSP description.

$$\texttt{EAST} = (\texttt{arrive} \rightarrow \texttt{EAST}).$$

$$\texttt{WEST} = (\texttt{depart} \rightarrow \texttt{WEST}).$$

$$
\begin{aligned}
&\texttt{CONTROL} = \texttt{CLOSED},\\
&\\
&\texttt{CLOSED} = (\texttt{open} \rightarrow \texttt{OPEN}),\\
&\\
&\texttt{OPEN} = (\\
&\quad \texttt{arrive} \rightarrow \texttt{OPEN}\\
&\quad \mid \texttt{depart} \rightarrow \texttt{OPEN}\\
&\quad \mid \texttt{close} \rightarrow \texttt{CLOSING}\\
&).\\
&\\
&\texttt{CLOSING} = (\texttt{depart} \rightarrow \texttt{CLOSING}).\\
&\\
&\texttt{DIRECTOR} = (\texttt{open} \rightarrow \texttt{close} \rightarrow \texttt{DIRECTOR}).\\
&\\
&\texttt{MUSEUM} = (\texttt{EAST} \parallel \texttt{WEST} \parallel \texttt{CONTROL} \parallel \texttt{DIRECTOR}).
\end{aligned}
$$

**2.b)** Model the above scenario with any valid Petri net.

**Solution:**

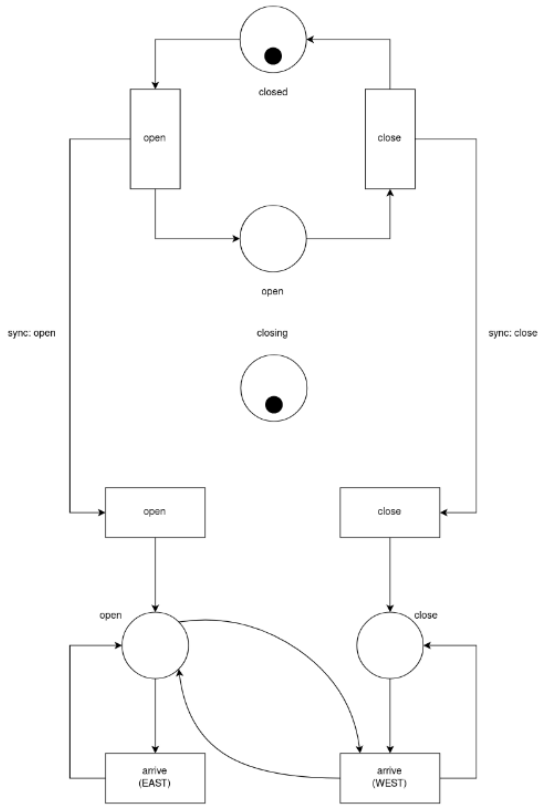


Figure 5: Structure diagram for the `MUSEUM` system.

**2.c)** Provide the Java classes that implement each one of the above FSP processes.

## Solution:

Listing 2: Concurrent system for museum entry and exit control

```
class MuseumController {
    private boolean open = false;

    public synchronized void openMuseum() {
        open = true;
        notifyAll();
    }

    public synchronized void closeMuseum() {
        open = false;
        notifyAll();
    }

    public synchronized void enter() throws InterruptedException {
        while (!open) {
            wait();
        }
```

```
    }

    public synchronized void exit() throws InterruptedException {
        while (!open && Thread.activeCount() > 2) {
            wait();
        }
    }
}

class Director extends Thread {
    private final MuseumController controller;

    public Director(MuseumController controller) {
        this.controller = controller;
    }

    @Override
    public void run() {
        try {
            controller.openMuseum();
            Thread.sleep(1000);
            controller.closeMuseum();
        } catch (InterruptedException e) {
            return;
        }
    }
}

class Visitor extends Thread {
    private final MuseumController controller;

    public Visitor(MuseumController controller) {
        this.controller = controller;
    }

    @Override
    public void run() {
        try {
            controller.enter();
            Thread.sleep((int) (Math.random() * 1000));
            controller.exit();
        } catch (InterruptedException e) {
            return;
        }
    }
}

public class MuseumSystem {
    public static void main(String[] args) throws InterruptedException {
        MuseumController controller = new MuseumController();
        Director director = new Director(controller);
        director.start();

        for (int i = 0; i < 5; i++) {
            new Visitor(controller).start();
            Thread.sleep(200);
        }
    }
}
```

## Question 3

Specify a safety property for the car park problem detailed in lecture notes 7 or chapter 5 of the textbook, which asserts that the car park does not overflow. Additionally, specify a progress property which asserts that cars eventually enter the car park. If car departure is a lower priority than car arrival, does starvation occur?

**Solution:**

**(I)** Specify safety and progress properties.

The property:

$$
\begin{aligned}
&\texttt{property PREVENTS\_OVERFLOW} = \texttt{SPOTS}[0] \\
&\\
&\texttt{SPOTS}[i:0..N] = ( \\
&\quad \texttt{when}(i < N)\ \texttt{arrive} \rightarrow \texttt{SPOTS}[i+1] \\
&\quad \mid \texttt{when}(i > 0)\ \texttt{depart} \rightarrow \texttt{SPOTS}[i-1] \\
&\quad \mid \texttt{when}(i == N)\ \texttt{arrive} \rightarrow \texttt{ERROR} \\
&).
\end{aligned}
$$

is a safety property for the car park system that prevents the car park from overflowing. Furthermore, the properties:

$$
\begin{aligned}
&\texttt{property WILL\_ARRIVE} = \{\texttt{arrive}\} \\
&\texttt{property WILL\_DEPART} = \{\texttt{depart}\}
\end{aligned}
$$

ensures that cars eventually arrive at the car park.

**(II)** Verify if starvation occurs.

If departures are of a lower priority than arrivals, neither the `arrive` nor `depart` actions starve. This is due to the fact that when the parking‘lot reaches capacity by continuing to allow arrivals, since it is of a higher priority, the only action which can be performed is the `depart` action, and once that is performed, then car park has a spot available and can thus allow but the `arrive` and `depart` action. Therefore, neither action starves.

## Question 4

For the dining philosophers problem, the act of simultaneously grabbing both forks is an abstraction of the general rule that the act of picking up both forks is atomic. In other words, the order in which the forks are picked up (left or right first) is arbitrary, but once the process of picking starts, it cannot be interrupted. In practice, conceptual simultaneity is often implemented as atomicity.

Provide an FSP for the dining philosophers problem where the act of sequentially picking up both forks is atomic.

**Solution:**

$$
\begin{aligned}
&\texttt{FORK} = (\\
&\quad \texttt{request\_right} \rightarrow \texttt{take\_right} \rightarrow \texttt{put\_right} \rightarrow \texttt{FORK}\\
&\quad | \ \texttt{request\_left} \rightarrow \texttt{take\_left} \rightarrow \texttt{put\_left} \rightarrow \texttt{FORK}\\
&).\\
\\
&\texttt{PHIL} = (\texttt{think} \rightarrow \texttt{request\_forks} \rightarrow \texttt{GRAB\_FORKS}).\\
\\
&\texttt{GRAB\_FORKS} = (\\
&\quad \texttt{take\_right} \rightarrow \texttt{take\_left} \rightarrow \texttt{eat} \rightarrow \texttt{DROP\_FORKS}\\
&\quad | \ \texttt{take\_left} \rightarrow \texttt{take\_right} \rightarrow \texttt{eat} \rightarrow \texttt{DROP\_FORKS}\\
&).\\
\\
&\texttt{DROP\_FORKS} = (\\
&\quad \texttt{put\_left} \rightarrow \texttt{put\_right} \rightarrow \texttt{PHIL}\\
&\quad | \ \texttt{put\_right} \rightarrow \texttt{put\_left} \rightarrow \texttt{PHIL}\\
&).\\
\\
&\texttt{DINERS}(N = 5) = (\\
&\quad \texttt{forall}[i : 1..N](\\
&\quad\quad \texttt{phil}[i] : \texttt{PHIL} \ || \ \{\texttt{phil}[i].\texttt{right}, \ \texttt{phil}[(i+1)\%N].\texttt{left}\} :: \texttt{FORK}\\
&\quad )\\
&)\{\\
&\quad \texttt{request\_forks/right.request\_right},\\
&\quad \texttt{request\_forks/left.request\_left},\\
&\quad \texttt{request\_forks\_1/request\_right\_1},\\
&\quad \texttt{request\_forks\_1/request\_left\_1},
\end{aligned}
$$

```
    request_forks_2/request_right_2,
    request_forks_2/request_left_2,
    request_forks_3/request_right_3,
    request_forks_3/request_left_3,
    request_forks_4/request_right_4,
    request_forks_4/request_left_4,
    request_forks_5/request_right_5,
    request_forks_5/request_left_5
}.
```

## Question 5

Provide any kind of Petri net representation of the asymmetric dining philosophers' problem discussed in lecture notes 9 or chapter 6 of the textbook.

**Solution:**

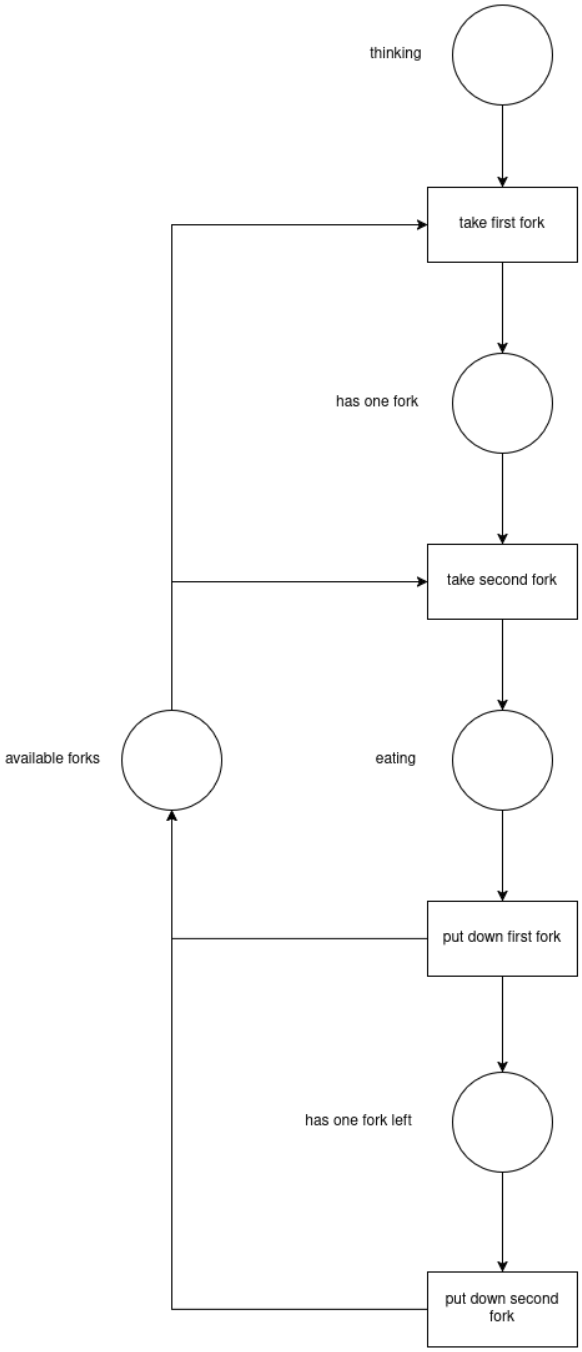


Figure 6: Petri net representation of the dining philosophers' problem.

## Question 6

A lift has a maximum capacity of ten people. In the model of the lift control system, passengers entering the lift are signalled by an enter action and passengers exiting the lift are signalled by an exit action. Specify a safety property as an FSP which, when composed with the lift, will check that the system never allows the lift it controls to have more than ten occupants.

**Solution:**

$$\texttt{property} \quad \texttt{MAX\_OCCUPANCY} = \texttt{occ}[0]$$

$$\begin{aligned}
&\texttt{occ}[i : 0..10] = ( \\
&\quad \texttt{enter} \rightarrow (\texttt{when}\ (i < 10)\ \texttt{occ}[i+1] \mid \texttt{when}\ i == 10\ \texttt{ERROR}) \\
&\quad \mid \texttt{exit} \rightarrow (\texttt{when}\ (i > 0)\ \texttt{occ}[i-1] \mid \texttt{when}\ i == 0\ \texttt{occ}[0]) \\
&)
\end{aligned}$$

## Question 7

Consider the following safety property:

$$\texttt{property P} = \texttt{(a} \rightarrow \texttt{(b} \rightarrow \texttt{P | a} \rightarrow \texttt{P) | b} \rightarrow \texttt{a} \rightarrow \texttt{P)}$$

Provide an LTS generated by the property P and a standard process SP such that LTS(P) = LTS(SP).

**Solution:**

By decomposing the nested processes in P, we can create the following standard process SP, which has an equivalent LTS to LTS(P):

$$\begin{aligned}
\texttt{SP} &= \texttt{S}_0 \\
\texttt{S}_0 &= (\texttt{a} \rightarrow \texttt{S}_1 \mid \texttt{b} \rightarrow \texttt{S}_2) \\
\texttt{S}_1 &= (\texttt{a} \rightarrow \texttt{S}_0 \mid \texttt{b} \rightarrow \texttt{S}_0) \\
\texttt{S}_2 &= (\texttt{a} \rightarrow \texttt{S}_0)
\end{aligned}$$

Graphically, the LTS of SP looks like:

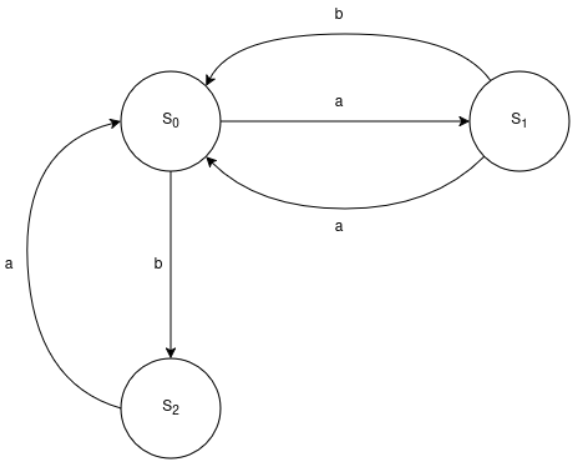


Figure 7: LTS of SP

## Question 8

Consider both the smokers' problem and the dining philosophers' problem presented in lecture notes 10 and 9 respectively.

**8.a)** Provide a straightforward FSP model of smokers' similar to that of the dining philosophers' problem. Add a supplier described on page 6 and represent the system using an FSP. Use both the compact and expanded FSP notation. For example, the tobacco smoker could be modelled by the process:

$$\texttt{SMOKER\_T} = (\texttt{get\_paper} \rightarrow \texttt{roll\_cigarette} \rightarrow \texttt{smoke\_cigarette} \rightarrow \texttt{SMOKER\_T})$$

The tobacco resource could be modelled by the process:

$$\texttt{TOBACCO} = (\texttt{delivered} \rightarrow \texttt{picked} \rightarrow \texttt{TOBACCO})$$

If your solution deadlocks, provide the shortest trace that leads to the deadlock. If it doesn't provide some arguments as to why.

**Solution:**

**(IV)** Compact FSP of the smokers' problem.

Listing 3: Compact FSP specification of smokers' problem

```
SMOKER_T = (
    get_paper -> get_match -> roll_cigarrette -> smoke_cigarrette ->
        SMOKER_T
).

SMOKER_M = (
    get_tobacco -> get_match -> roll_cigarrette -> smoke_cigarrette ->
        SMOKER_P
).

SMOKER_P = (
    get_tobacco -> get_paper -> roll_cigarrette -> smoke_cigarrette ->
        SMOKER_M
).

TOBACCO = (delivered -> picked -> TOBACCO).
PAPER   = (delivered -> picked -> PAPER).
MATCH   = (delivered -> picked -> MATCH).

AGENT = (can_deliver -> deliver_ing1 -> deliver_ing2 -> AGENT).

RULE = (can_deliver -> finish_smoking -> RULE).

SMOKERS = (st:SMOKER_T || sp:SMOKER_P || sm:SMOKER_M).

RESOURCES = (
    {sm, sp} :: TOBACCO ||
```

```
    {st, sm} :: PAPER   ||
    {st, sp} :: MATCH
).

AGENTS_RULE = (
    {st, sp, sm} :: RULE ||
    {sm, sp} :: AGENT ||
    {sm, st} :: AGENT ||
    {st, sp} :: AGENT
).

SYSTEM = (SMOKERS || RESOURCES || AGENTS_RULE) / {
    st.get_paper/st.picked, st.get_match/st.picked,
    sp.get_tobacco/sp.picked, sp.get_match/sp.picked,
    sm.get_tobacco/sm.picked, sm.get_paper/sm.picked,
    st.deliver_ing1/st.delivered, st.deliver_ing2/st.delivered,
    sp.deliver_ing1/sp.delivered, sp.deliver_ing2/sp.delivered,
    sm.deliver_ing1/sm.delivered, sm.deliver_ing2/sm.delivered,
    st.finish_smoking/st.smoke_cigarrette,
    sp.finish_smoking/sp.smoke_cigarrette,
    sm.finish_smoking/sm.smoke_cigarrette
}.
```

## (V) Expanded FSP.

Listing 4: Expanded FSP specification of smokers' problem

```
TOBACCO = (delivered -> picked -> TOBACCO).
PAPER   = (delivered -> picked -> PAPER).
MATCH   = (delivered -> picked -> MATCH).

SMOKER_T = (get_paper -> get_match -> roll_cigarrette ->
    smoke_cigarrette -> SMOKER_T).
SMOKER_M = (get_tobacco -> get_match -> roll_cigarrette ->
    smoke_cigarrette -> SMOKER_P).
SMOKER_P = (get_tobacco -> get_paper -> roll_cigarrette ->
    smoke_cigarrette -> SMOKER_M).

AGENT_T = (can_deliver -> deliver_paper -> deliver_match ->
    AGENT_T).
AGENT_P = (can_deliver -> deliver_match -> deliver_tobacco ->
    AGENT_P).
AGENT_M = (can_deliver -> deliver_tobacco -> deliver_paper ->
    AGENT_M).

RULE = (can_deliver -> finish_smoking -> RULE).

SMOKERS = (st:SMOKER_T || sp:SMOKER_P || sm:SMOKER_M).

RESOURCES = (
    {sm, sp} :: TOBACCO ||
    {st, sm} :: PAPER   ||
    {st, sp} :: MATCH
).

AGENTS = (
    {sm, sp} :: AGENT_T ||
    {sm, st} :: AGENT_P ||
```

```
    {st, sp} :: AGENT_M
).

COORDINATION = (
    {st, sp, sm} :: RULE
).

SYSTEM = (SMOKERS || RESOURCES || AGENTS || COORDINATION) / {
    st.get_paper/st.picked, st.get_match/st.picked,
    sp.get_tobacco/sp.picked, sp.get_match/sp.picked,
    sm.get_tobacco/sm.picked, sm.get_paper/sm.picked,
    st.deliver_paper/st.delivered, st.deliver_match/st.delivered,
    sp.deliver_tobacco/sp.delivered, sp.deliver_match/sp.delivered
        ,
    sm.deliver_tobacco/sm.delivered, sm.deliver_paper/sm.delivered
        ,
    st.finish_smoking/st.smoke_cigarrette,
    sp.finish_smoking/sp.smoke_cigarrette,
    sm.finish_smoking/sm.smoke_cigarrette
}.
```

**(VI)** Deadlock analysis.

The solution does not exhibit deadlock. This can be verified by analyzing the behavior and synchronization of the defined FSP processes. The `RULE` process enforces coordination among all components by ensuring that new ingredients are not delivered until the current smoking cycle is complete. Specifically, it requires that a `finish_smoking` action occur before another `can_deliver` action is permitted. This guarantees that only one set of ingredients is ever present on the table at a time, preventing simultaneous deliveries and eliminating resource conflicts.

Each smoker process also follows a fixed, sequential pattern: `get_ingredient1` → `get_ingredient2` → `roll_cigarrette` → `smoke_cigarrette`. After the smoking phase, the smoker triggers `finish_smoking`, which signals to the `RULE` process that the cycle has completed and allows the next delivery to proceed. This strict sequence ensures that no smoker can progress out of order or hold resources indefinitely.

Furthermore, the system enforces mutual exclusion among agents. Since all agents synchronize on the `can_deliver` event, only one agent can perform a delivery at any given time. The `RULE` process governs this synchronization, authorizing exactly one delivery before requiring the corresponding `finish_smoking` event. As a result, each cycle involves precisely one agent delivering one valid pair of ingredients.

Finally, no circular wait conditions can occur. Each smoker only waits for the specific ingredients they are missing, meaning there is no shared contention over the same resources. Only one smoker obtains both required ingredients in any given cycle, and the process `(smoke_cigarrette` → `finish_smoking` → `can_deliver)` always completes fully before another cycle begins.

Therefore, this FSP specification successfully models the Smokers Problem without deadlock. The interaction between the `RULE`, `AGENT`, and `SMOKER` processes ensures that all participants take turns according to the resources available, allowing the system to continuously and safely progress through the `deliver` → `pick` → `smoke` → `deliver` cycle.

**8.b)** Write a safety property as a process with the syntax:

```
property CORRECT_PICKUP = ...
```

that verifies the correct sequences of resource picking. Compose `CORRECT_PICKUP` with your solution in (a) and use the system provided by the textbook to verify that the safety property isn't violated.

**Solution:**

Listing 5: Property and composed process for correct pickup

```
property CORRECT_PICKUP = (
    st.get_paper -> st.get_match -> CORRECT_PICKUP
  | sp.get_tobacco -> sp.get_match -> CORRECT_PICKUP
  | sm.get_tobacco -> sm.get_paper -> CORRECT_PICKUP
).

FULL_CIG_SMOKERS = (SMOKERS || RESOURCES || AGENT_RULE || CORRECT_PICKUP)
    / {
    st.get_paper/st.picked, sm.get_paper/sm.picked, sp.get_paper/sp.picked
        ,
    st.deliver_paper/st.delivered, sm.deliver_paper/sm.delivered, sp.
        deliver_paper/sp.delivered,
    st.smoking_completed/st.smoke_cigarrette,
    sm.smoking_completed/sm.smoke_cigarrette,
    sp.smoking_completed/sp.smoke_cigarrette
}.
```

**8.c)** An elegant, deadlock-free solution to the smokers problem can be constructed by applying an "ask first, do later" approach. Assume that the smokers look at the table first, before making a move. Then each smoker can start requesting resources only if he has the ingredient he does not see on the table, otherwise, the smoker waits. Provide this solution usings FSPs.

**Solution:**

Listing 6: FSP model of cigarette smokers problem

```
SMOKER_T = (no_tobacco -> get_paper -> get_match -> roll_cigarrette ->
    smoke_cigarrette -> SMOKER_T).
SMOKER_P = (no_paper -> get_tobacco -> get_match -> roll_cigarrette ->
    smoke_cigarrette -> SMOKER_P).
SMOKER_M = (no_match -> get_tobacco -> get_paper -> roll_cigarrette ->
    smoke_cigarrette -> SMOKER_T).

TOBACCO = (delivered -> picked -> TOBACCO).
PAPER   = (delivered -> picked -> PAPER).
MATCH   = (delivered -> picked -> MATCH).

AGENT_T = (can_deliver -> no_tobacco -> deliver_paper -> deliver_match ->
    AGENT_T).
AGENT_P = (can_deliver -> no_paper -> deliver_match -> deliver_tobacco ->
    AGENT_P).
```

```
AGENT_M = (can_deliver -> no_match -> deliver_tobacco -> deliver_paper ->
    AGENT_M).

RULE = (can_deliver -> smoking_completed -> RULE).

SMOKERS = (st:SMOKER_T || sp:SMOKER_P || sm:SMOKER_M).

RESOURCES = (
    {sm, sp} :: TOBACCO ||
    {st, sm} :: PAPER   ||
    {st, sp} :: MATCH
).

AGENT_RULE = (
    {sm, sp, st} :: RULE ||
    {sm, sp}     :: AGENT_T ||
    {sm, st}     :: AGENT_P ||
    {st, sp}     :: AGENT_M
).

CIG_SMOKERS = (SMOKERS || RESOURCES || AGENT_RULE) / {
    st.get_paper/st.picked, sm.get_paper/sm.picked, sp.get_paper/sp.picked
        ,
    st.deliver_paper/st.delivered, sm.deliver_paper/sm.delivered, sp.
        deliver_paper/sp.delivered,
    st.smoking_completed/st.smoke_cigarrette,
    sm.smoking_completed/sm.smoke_cigarrette,
    sp.smoking_completed/sp.smoke_cigarrette
}.
```

**8.d)** Compose your solution from (c) with the `CORRECT_PICKUP` property from (b) and use the system provided by the textbook to verify that the safety property is not violated.

**Solution:**

Using the following LTSA code:

```
// Cigarette smokers system + CORRECT_PICKUP property
SMOKER_T = (no_tobacco -> get_paper -> get_match ->
    roll_cigarrette -> smoke_cigarrette -> SMOKER_T).
SMOKER_P = (no_paper -> get_tobacco -> get_match ->
    roll_cigarrette -> smoke_cigarrette -> SMOKER_P).
SMOKER_M = (no_match -> get_tobacco -> get_paper ->
    roll_cigarrette -> smoke_cigarrette -> SMOKER_T).

TOBACCO = (delivered -> picked -> TOBACCO).
PAPER   = (delivered -> picked -> PAPER).
MATCH   = (delivered -> picked -> MATCH).

AGENT_T = (can_deliver -> no_tobacco -> deliver_paper ->
    deliver_match -> AGENT_T).
AGENT_P = (can_deliver -> no_paper -> deliver_match ->
    deliver_tobacco -> AGENT_P).
AGENT_M = (can_deliver -> no_match -> deliver_tobacco ->
    deliver_paper -> AGENT_M).
```

```
RULE = (can_deliver -> smoking_completed -> RULE).

SMOKERS = (st:SMOKER_T || sp:SMOKER_P || sm:SMOKER_M).

RESOURCES = (
    {sm, sp} :: TOBACCO ||
    {st, sm} :: PAPER   ||
    {st, sp} :: MATCH
).

AGENT_RULE = (
    {sm, sp, st} :: RULE ||
    {sm, sp}     :: AGENT_T ||
    {sm, st}     :: AGENT_P ||
    {st, sp}     :: AGENT_M
).

CIG_SMOKERS = (SMOKERS || RESOURCES || AGENT_RULE) / {
    st.get_paper/st.picked, sm.get_paper/sm.picked, sp.get_paper/
        sp.picked,
    st.deliver_paper/st.delivered, sm.deliver_paper/sm.delivered,
        sp.deliver_paper/sp.delivered,
    st.smoking_completed/st.smoke_cigarrette,
    sm.smoking_completed/sm.smoke_cigarrette,
    sp.smoking_completed/sp.smoke_cigarrette
}.

property CORRECT_PICKUP = (
    st.get_paper -> st.get_match -> CORRECT_PICKUP
  | sp.get_tobacco -> sp.get_match -> CORRECT_PICKUP
  | sm.get_tobacco -> sm.get_paper -> CORRECT_PICKUP
).

FULL_CIG_SMOKERS = (SMOKERS || RESOURCES || AGENT_RULE ||
    CORRECT_PICKUP) / {
    st.get_paper/st.picked, sm.get_paper/sm.picked, sp.get_paper/
        sp.picked,
    st.deliver_paper/st.delivered, sm.deliver_paper/sm.delivered,
        sp.deliver_paper/sp.delivered,
    st.smoking_completed/st.smoke_cigarrette,
    sm.smoking_completed/sm.smoke_cigarrette,
    sp.smoking_completed/sp.smoke_cigarrette
}.
```

We can indeed verify that the `CORRECT_PICKUP` property is not violated by the FSP.